Predicting LLM Robustness Collapse from Tokenization Fragmentation under Typographical Noise

Yassine El Zeort, Mira Kamal, Kareen Terkawi
Department of Computer Science and Mathematics, Lebanese American University, Beirut, Lebanon
Emails: {yassine.elzeort, mira.kamal, kareen.terkawi}@lau.edu

***Abstract*—Large language models deployed in production fail unpredictably on typographically noisy input, yet failure is observable only after the full forward pass completes—no existing work attempts to predict it beforehand. We propose a pre-inference failure prediction framework that exploits tokenization fragmentation, computed from the tokenizer alone in under 5 ms on CPU, as a supervised signal to intercept high-risk inputs before they reach the LLM. The system operates in two temporally disjoint stages. Stage A constructs a structured per-instance robustness dataset by running Mistral-7B-Instruct v0.2 on clean and perturbed variants of SQuAD v2.0, SST-2, and GSM8K under six typographical noise families at three severity levels, producing approximately 54,000 labeled pairs with a failure threshold of 20% absolute performance drop. Stage B trains lightweight classifiers—Logistic Regression, Random Forest, and XGBoost—on a five-dimensional tokenization feature vector to output a calibrated failure probability that gates LLM access; inputs exceeding a learned risk threshold trigger safeguards before any GPU resource is consumed. We target macro F1 > 0.75 for classification and $R^2 > 0.70$ for severity regression, and evaluate cross-condition generalization across all six noise families and three task domains. To our knowledge, this is the first work to frame tokenization fragmentation as a supervised pre-inference failure signal and the first to propose a tokenizer-based input screening architecture for LLM deployment.**

***Index Terms*—large language models, tokenization fragmentation, typographical noise, pre-inference failure prediction, robustness**

I. Introduction

Large language models (LLMs) such as GPT-4, LLaMA, and Mistral have become foundational components in real-world natural language processing applications, including open-domain question answering, virtual assistants, and instruction-following agents [1], [2]. These systems are almost universally designed under an implicit assumption that is routinely violated in practice: that user-supplied text will be clean, correctly spelled, and lexically well-formed. In reality, character-level typographical errors—keystrokes missed, transposed, inserted, or substituted—represent the normal condition of human text input, arising from mobile keyboards, rapid typing, and autocorrect failures.

A substantial body of evidence demonstrates that such perturbations cause significant performance degradation across question answering, sentiment classification, and mathematical reasoning tasks [1], [2], [6], [7], yet the dominant inference architecture forwards every input unconditionally to the full inference stack—failure is discovered only after the entire computational expenditure is irrecoverable.

The consequences of this reactive architecture extend well beyond answer correctness. LLM inference is among the most energy-intensive operations in modern computing. Patterson et al. quantified the energy use and carbon footprint of training several large neural networks [11],, and Luccioni et al. demonstrated that inference—not training—constitutes the dominant ongoing energy cost at production throughput [12]. At production scale, even a modest interception rate eliminates hundreds of thousands of wasted GPU-seconds daily. A lightweight pre-inference screening mechanism capable of predicting LLM failure before any GPU resource is consumed would directly operationalize this principle, converting a reactive failure-discovery problem into a proactive and resource-conserving one.

Prior work on LLM robustness under typographical noise is well-established but uniformly reactive. Wang *et al.* [1] and Alahmari *et al.* [2] characterize the magnitude of performance drop across multiple models and noise types, yet make no attempt to predict failure before inference is initiated. Gan *et al.* [6] demonstrate that reasoning tasks are especially brittle under adversarial typographical perturbations, while Abedin *et al.* [7] report analogous vulnerability in mathematical problem solving. The most directly relevant prior work is Chai *et al.* [3], who systematically demonstrate that subword tokenization is sensitive to character-level noise—what they term the "curse of tokenization"—and identify fragmentation as a structural mechanism underlying LLM fragility. Their analysis, however, remains diagnostic: fragmentation is described as a phenomenon but is neither quantified as a predictive feature nor used to construct any pre-inference intervention. Zheng *et al.* [5] present the strongest counterargument, claiming that modern LLMs internally compensate for non-canonical tokenizations, which motivates rigorous empirical testing of the fragmentation-as-predictor hypothesis. Despite this extensive body of work, no prior study has framed tokenization fragmentation as a supervised pre-inference failure signal, and no prior work proposes an input gating architecture that screens inputs before LLM inference is initiated.

This paper makes three contributions: (1) a structured per-instance robustness dataset pairing fragmentation features with LLM performance labels, (2) a lightweight pre-inference failure predictor operating exclusively on tokenizer output, and (3) a cross-condition generalization analysis across task domains and noise families.

II. Literature Review

A. Robustness Evaluation Papers

Wang et al. [1] inject five types of noise—including typographical errors and OCR errors—into the MMLU benchmark and evaluate three LLMs, finding accuracy drops of up to 8.2% and noting that character-level errors are particularly damaging because they disrupt tokenization. They propose a "re-pass" denoising strategy, but open-source models largely fail at self-correction, and the entire framework requires running the LLM before failure is known. Zhu et al. [17] similarly benchmark LLM robustness under adversarial prompt perturbations across multiple models.

Alahmari et al. [2], [20] evaluate six LLMs on keyboard-typo and word-replacement perturbations across two sentiment datasets, measuring output divergence via BLEU, ROUGE-L, and Jaccard Index. They observe that perturbed words are fragmented into more sub-tokens by the tokenizer, causing output inconsistency, yet they do not quantify fragmentation systematically or predict failure per input.

Liu et al. [4], [18] introduce MulTypo, a multilingual keyboard-aware typo generation algorithm, and evaluate 18 LLMs across five tasks in multiple languages. Their results confirm that larger models are more robust and that mathematical reasoning tasks are especially brittle under noise, but the study operates entirely in the post-inference measurement regime with no predictive component.

Gan et al. [6] design the Adversarial Typo Attack (ATA) algorithm to identify minimal character edits that maximally disrupt Chain-of-Thought reasoning, demonstrating that a single character edit drops Mistral-7B-Instruct accuracy on GSM8K from 43.7% to 38.6%. The method requires gradient access to a running LLM and produces no pre-inference failure signal. Abedin et al. [7] report analogous brittleness in mathematical reasoning under punctuation noise, with degradation scaling with noise level.

Ma et al. [8] formalize robustness against character-level perturbations for neural network language models using a formal verification framework based on convex hull approximation. While theoretically grounded, the work does not address LLMs, does not quantify tokenization fragmentation, and proposes no runtime screening mechanism. We include this work to demonstrate that even formal verification approaches to character-level robustness do not address the pre-inference prediction gap.

B. Tokenization Sensitivity Papers

Chai et al. [3] systematically study what they term the "curse of tokenization"—the sensitivity of subword tokenization to typos, length variation, and lack of internal token structure awareness—across 13 tasks on LLaMA3, Mistral, and GPT-4 families. They show that character-level noise causes greater degradation than token-level noise, and that BPE-dropout at moderate rates partially mitigates the issue. They identify tokenization as the failure mechanism but do not compute per-input fragmentation features or build any predictive model.

Bhatia et al. [9] demonstrate that BPE tokenizers fragment calendar dates into meaningless sub-tokens, introducing a date fragmentation ratio metric and showing accuracy drops of up to 10 points on uncommon dates across temporal reasoning tasks. Their work confirms that fragmentation correlates with performance drops, but is restricted to date expressions and does not generalize to arbitrary typographical noise or build a cross-task predictor.

Tsuji et al. [10] investigate which neurons and attention heads are responsible for handling typographical errors inside LLMs, finding that "typo neurons" across early, middle, and late layers collectively perform internal correction using local and global context. This mechanistic analysis explains how LLMs process typos internally but provides no method for predicting failure before inference.

C. The Counterargument

Zheng et al. [5] argue that instruction-tuned LLMs are surprisingly robust to non-canonical tokenizations unseen during training, with Qwen-2.5-7B-Instruct retaining 93.4% of its canonical performance under random re-segmentation across 20 benchmarks. This represents the strongest challenge to our hypothesis; however, their study uses vocabulary-aligned re-segmentations constructed algorithmically, not the real-world character-level noise that arises from user typos. Our work empirically tests whether natural typographical fragmentation still predicts LLM failure in a supervised setting.

D. Qualitative Comparison

Table I maps each prior work to the five core challenges this research addresses. C1: pre-inference failure detection; C2: per-input failure predictor; C3: fragmentation quantified as a feature; C4: cross-task generalization tested; C5: cross-noise-type generalization tested.

E. Comparative Analysis

Prior work has collectively established that LLM performance degrades under typographical noise and that tokenization fragmentation is a key contributing mechanism. However, every existing paper operates reactively, and no work builds a supervised model using per-input fragmentation metrics as predictive features or proposes a pre-inference screening architecture. This gap is precisely what our work fills.

III. Contributions

This work makes three contributions, each absent from the existing literature:

Contribution 1 — Structured Per-Instance Robustness Dataset. We construct the first dataset that pairs every noisy input sentence with its tokenization fragmentation feature vector and the observed LLM performance score. Existing robustness studies report only aggregate statistics over test sets; we expose per-instance $(\phi, \Delta P)$ pairs across three tasks, six noise families, and three severity levels, producing approximately 54,000 labeled instances in a format directly usable for supervised learning.

Contribution 2 — Pre-Inference Failure Predictor. We train and validate the first supervised ML model that predicts

TABLE I
QUALITATIVE COMPARISON OF PRIOR WORK AGAINST THE FIVE CHALLENGES ADDRESSED BY THIS RESEARCH.

Paper	C1	C2	C3	C4	C5	Venue
Wang et al. [1]	No	No	No	Partial	No	EMNLP 2024
Alahmari et al. [2]	No	No	No	No	No	Sci. Reports 2026
Chai et al. [3]	No	No	Partial	No	No	EMNLP 2024
Liu et al. [4]	No	No	No	Partial	Partial	arXiv 2025
Zheng et al. [5]	No	No	Partial	No	No	arXiv 2025
Gan et al. [6]	No	No	No	No	No	EMNLP 2024
Abedin et al. [7]	No	No	No	No	No	arXiv 2025
Bhatia et al. [9]	No	No	Yes	No	No	EMNLP 2025
Tsuji et al. [10]	No	No	Partial	No	No	EMNLP 2025
Ma et al. [8]	No	No	No	No	No	ICFEM 2023
Our Method	**Yes**	**Yes**	**Yes**	**Yes**	**Yes**	**This paper**

LLM failure probability from tokenization fragmentation features alone, before any inference is performed. The classifier operates in under 5 ms on CPU using a five-dimensional feature vector computed exclusively from the tokenizer, enabling a production-ready gating architecture that intercepts high-risk inputs before GPU resources are consumed. To our knowledge, no prior work frames fragmentation as a supervised pre-inference failure signal.

Contribution 3 — Cross-Condition Generalization Analysis. We evaluate whether the fragmentation–failure relationship generalizes across task domains (QA, sentiment, reasoning) and noise families (keyboard-proximity, homoglyph, repetition, deletion, whitespace, case). This cross-condition test has never been performed; either outcome — generalizable signal or condition-specific behavior — advances the scientific understanding of tokenization-driven LLM fragility.

IV. SYSTEM MODEL

Large language model inference is inherently reactive: failure is observable only after the forward pass completes, by which point computational resources have been fully consumed. We propose a proactive pre-inference screening mechanism that intercepts high-risk inputs before they reach the LLM. The framework operates in two temporally disjoint stages, illustrated in Figure 1.

Stage A executes offline, once, to construct a supervised training dataset by empirically characterizing the relationship between tokenization fragmentation and LLM performance degradation. Stage B deploys at inference time as a lightweight gating function operating exclusively on tokenizer output, with zero LLM involvement.

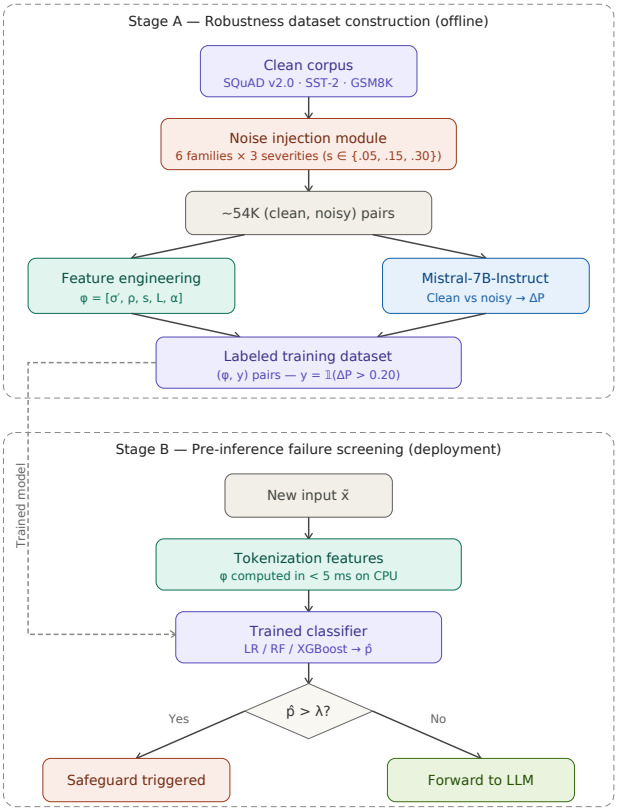

Fig. 1. Two-stage pre-inference failure prediction system. Stage A constructs the robustness dataset offline; Stage B screens each new input at deployment time before any LLM inference is performed.

A. Stage A: Training Dataset Construction

For each clean English sentence x, a noise injection module generates perturbed variants $\tilde{x}$ by applying one of six noise families at severity levels $s \in \{0.05, 0.15, 0.30\}$, producing up to 18 noisy variants per sentence (Section VB). For each pair $(x, \tilde{x})$, the tokenizer is applied independently to both versions and a five-dimensional feature vector $\boldsymbol{\phi} \in \mathbb{R}^5$ is computed in under 5 ms on CPU (Section VC). The target LLM M then processes both versions under standardized zero-shot prompts (Section VD) to produce task performance scores P_{clean} and P_{noisy}. The performance drop is:

$$\Delta P = P_{\text{clean}} - P_{\text{noisy}} \in [0, 1] \tag{1}$$

A binary label is assigned: y=1 (FAILURE) if $\Delta P > \theta$, and y=0 (ROBUST) otherwise, where θ=0.20 (Section VE). The continuous value P_{noisy} is retained as a secondary regression target encoding failure severity. Stage A produces dataset $\mathcal{D} = \{(\boldsymbol{\phi}_i, y_i, P_{\text{noisy},i})\}_{i=1}^{N}$, partitioned 80/20 using stratified splitting to preserve class balance.

Parameters. $|\mathcal{D}| \approx 54{,}000$ pairs across 3 tasks, 6 noise types, and 3 severity levels. Feature dimensionality: d=5. Failure threshold: θ=0.20.

B. Stage B: Pre-Inference Failure Gating

At deployment time, a new input $\tilde{x}$ arrives. The system computes $\boldsymbol{\phi}$ from the tokenizer alone in under 5 ms — all five features are computed exclusively from $\tilde{x}$ and corpus-level constants derived during training, requiring no clean reference. The trained classifier produces a failure probability:

$$\hat{p} = f_{\text{clf}}(\boldsymbol{\phi}) \in [0, 1] \tag{2}$$

A routing decision is made via threshold λ, selected on the validation set by maximizing F1:

$$\text{Route}(\tilde{x}) = \begin{cases} \textsc{Fail} & \hat{p} > \lambda \\ \textsc{Robust} & \hat{p} \leq \lambda \end{cases} \tag{3}$$

Under ROBUST, the input proceeds to the LLM. Under FAIL, the input is intercepted before any inference cost is incurred and a safeguard is triggered — spelling normalization for moderate-risk inputs ($\hat{p} \in (\lambda, 0.85]$), or user clarification / fallback model routing for high-risk inputs ($\hat{p} > 0.85$) [2]. A secondary regressor f_{reg} estimates $\hat{a} = f_{\text{reg}}(\boldsymbol{\phi}) \in [0, 1]$, quantifying failure severity for downstream safeguard prioritization. To our knowledge, no prior work deploys a supervised ML model using tokenization fragmentation as the sole feature set for pre-inference LLM failure prediction.

V. Proposed Approach

A. Benchmark Datasets

We sample from three English benchmarks spanning distinct reasoning regimes. Following established robustness evaluation protocols [1], [3], we adopt: **SQuAD v2.0** (reading comprehension; metrics: F1 and Exact Match), **SST-2** (binary sentiment classification; metric: accuracy), and **GSM8K** (multi-step mathematical reasoning; metric: final answer accuracy). SST-2 serves as a scientifically necessary negative contrast where fragmentation is expected to be a weaker predictor, while GSM8K targets the strongest fragmentation–failure correlation [6], [7].

We sample 1,000 clean sentences per dataset (3,000 total), producing approximately 54,000 (clean, noisy) pairs after noise injection. This comfortably exceeds the sample size at which tree ensembles reach performance saturation on low-dimensional tabular problems [15]. This work is scoped to English; multilingual extension is left as future work.

B. Noise Injection

We design a six-family noise injection module covering the typographical failure space of real deployment environments [1], [2]. Each family is applied independently at severity $s \in \{0.05, 0.15, 0.30\}$, yielding $6 \times 3 = 18$ variants per sentence. Severity is a character-level perturbation rate; the absolute number of corrupted characters scales with sentence length L, controlled for by including L in the feature vector. Perturbation positions are sampled uniformly with fixed seed r=42. Table II summarizes all six families.

TABLE II
SIX NOISE FAMILIES USED FOR TYPOGRAPHICAL PERTURBATION.

Noise Family	Example	Simulates
Keyboard-proximity substitution	network → netqork	Adjacent-key slip [13]
Homoglyph substitution	role → r0le	Unicode looka-likes [3], [4]
Character repetition	address → adddress	Key-hold errors
Character deletion	capital → caital	Missed keystroke
Intra-word whitespace	context → con text	BPE boundary exploit [3]
Random case flip	Mistral → mIsTrAl	Case-sensitive vocab disruption [4]

C. Tokenization Feature Engineering

Each input $\tilde{x}$ is represented by $\boldsymbol{\phi} = [\sigma', \rho, s, L, \alpha]^\top$, computed exclusively from $\tilde{x}$ and corpus-level constants. Let $T_n = |T(\tilde{x})|$ denote the token count under the Mistral BPE tokenizer, and V_{clean} the reference vocabulary from all clean training sentences. Feature independence is verified via the pairwise Pearson correlation matrix; any pair with $|r| > 0.85$ triggers removal of the lower-importance feature.

Tokens-per-word ratio $\sigma' = T_n/L$. The primary fragmentation signal, normalized by word count for cross-sentence comparability. Clean English produces $\sigma' \approx 1.2$–1.4; heavily noisy inputs exceed 2.0.

OOV token rate $\rho = |\{t \in T(\tilde{x}) \mid t \notin V_{\text{clean}}\}|/T_n$. Captures fragment *novelty* — orthogonal to σ', which captures quantity.

Noise severity s. The perturbation rate, known exactly during training. At deployment, s is approximated as the fraction of characters absent from the clean training character inventory. The practical impact of this approximation is assessed in Experiment 5.

Sentence length L (word count). Decouples the input-size confound from the fragmentation signal [1].

Single-character token rate $\alpha = |\{t \in T(\tilde{x}) : |t| = 1\}|/T_n$. Captures *BPE collapse* — when the tokenizer falls back to character-level segmentation under severe noise, a qualitatively different phenomenon from high σ' or ρ.

D. Target LLM and Prompting Protocol

We select Mistral-7B-Instruct v0.2 [14] as the target LLM, satisfying four constraints: (i) open weights (Apache 2.0) for full reproducibility; (ii) publicly accessible BPE tokenizer via HuggingFace `AutoTokenizer`, a hard prerequisite for tokenization-based research; (iii) computational feasibility in 4-bit NF4 quantization on a free-tier Colab T4 GPU; and (iv) deployment representativeness as a competitive 7B instruction-following model [14].

All tasks are evaluated zero-shot with fixed prompt templates: SQuAD uses passage + question with regex-based span extraction; SST-2 uses exact string matching on "positive"/"negative"; GSM8K extracts the final number after "The answer is:". We adopt zero-shot prompting uniformly because

our study measures *relative* degradation ΔP under identical conditions, not absolute accuracy [16].

E. Failure Definition

An input is labeled FAILURE when the performance drop exceeds a threshold sufficient to alter output utility:

$$y = \begin{cases} 1\ (\text{FAILURE}) & \text{if } \Delta P > 0.20 \\ 0\ (\text{ROBUST}) & \text{if } \Delta P \leq 0.20 \end{cases} \tag{4}$$

Sensitivity analysis at $\theta \in \{0.10, 0.30\}$ assesses threshold stability.

F. Predictive Models

Classification (primary). Logistic Regression (ℓ_2 baseline), Random Forest, and XGBoost, all outputting $\hat{p} \in [0, 1]$ via `predict_proba()`. Hyperparameters are selected via 5-fold cross-validated grid search; XGBoost uses early stopping (patience 50, AUC-ROC criterion). All splits are stratified. Target: macro F1 > 0.75.

Regression (secondary). Random Forest and XGBoost regressors predict P_{noisy} as a continuous severity estimate. Target: $R^2 > 0.70$.

Tree ensembles are selected over deep alternatives because d=5 features would severely overparameterize neural networks; tree methods additionally provide permutation-based feature importance scores central to validating our feature engineering rationale [15].

VI. Plan Forward

A. Implementation

The pipeline is implemented in Python 3.10 using HuggingFace `datasets`, `transformers`, and `AutoTokenizer`. The reference vocabulary V_{clean} is constructed by tokenizing all clean training sentences prior to feature computation. LLM inference runs on Google Colab (T4, free tier) with Mistral-7B-Instruct v0.2 in 4-bit NF4 quantization. Classifiers are implemented via `scikit-learn` and `xgboost`, with all sampling under fixed seed r=42.

B. Experimental Protocol

Experiment 1 — Correlation Analysis. Pearson r and Spearman ρ between each feature and ΔP, stratified by task and noise family. The pairwise feature correlation matrix enforces the collinearity criterion before model training.

Experiment 2 — Classification Benchmark. All classifiers evaluated on the stratified held-out set. Metrics: macro F1, precision, recall, AUC-ROC, calibration curves, and false positive rate.

Experiment 3 — Regression Benchmark. Severity regressors evaluated on R^2, MAE, and rank-ordering across severity levels.

Experiment 4 — Cross-Condition Generalization. Classifiers trained on each task are tested on the remaining two; classifiers trained on each noise family are tested on the remaining five. Generalization ratio $\gamma = \text{F1}_{\text{cross}}/\text{F1}_{\text{in}}$ quantifies transferability.

Experiment 5 — Feature Ablation. Leave-one-out ablation reports ΔF1 per removed feature. The ablation of s includes a direct comparison of exact vs. estimated deployment-time values; if the gap exceeds 0.05 F1, s is excluded and the model retrained.

References

[1] J. Wang *et al.*, "Resilience of large language models for noisy instructions," in *Findings of ACL: EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.697.pdf

[2] S. S. Alahmari, L. Hall, P. R. Mouton, and D. Goldgof, "Large language models robustness against perturbation," *Scientific Reports*, vol. 15, 2026. [Online]. Available: https://www.nature.com/articles/s41598-025-29770-0

[3] Y. Chai, Y. Fang, Q. Peng, and X. Li, "Tokenization falling short: On subword robustness in large language models," in *Findings of ACL: EMNLP 2024*, pp. 1582–1599, 2024. [Online]. Available: https://aclanthology.org/2024.findings-emnlp.86.pdf

[4] A. Liu *et al.*, "Evaluating robustness of large language models against multilingual typographical errors," *arXiv preprint arXiv:2510.09536*, 2025. [Online]. Available: https://arxiv.org/abs/2510.09536

[5] B. S. Zheng, A. Liu, O. Ahia, J. Hayase, Y. Choi, and N. A. Smith, "Broken tokens? Your language model can secretly handle non-canonical tokenizations," *arXiv preprint arXiv:2506.19004*, 2025. [Online]. Available: https://arxiv.org/abs/2506.19004

[6] E. Gan *et al.*, "Reasoning robustness of LLMs to adversarial typographical errors," in *Proc. EMNLP 2024*, 2024. [Online]. Available: https://aclanthology.org/2024.emnlp-main.584.pdf

[7] Z. U. Abedin *et al.*, "ArithmAttack: Evaluating robustness of LLMs to noisy context in math problem solving," *arXiv preprint arXiv:2501.08203*, 2025. [Online]. Available: https://arxiv.org/abs/2501.08203

[8] Z. Ma *et al.*, "Formalizing robustness against character-level perturbations for neural network language models," in *ICFEM 2023, LNCS*, vol. 14308, Springer, 2023. [Online]. Available: https://doi.org/10.1007/978-981-99-7584-6_7

[9] G. Bhatia, M. Peyrard, and W. Zhao, "Date fragments: A hidden bottleneck of tokenization for temporal reasoning," in *Proc. EMNLP 2025*, 2025. [Online]. Available: https://aclanthology.org/2025.emnlp-main.159.pdf

[10] K. Tsuji *et al.*, "Investigating neurons and heads in transformer-based LLMs for typographical errors," in *Proc. EMNLP 2025*, pp. 6145–6163, 2025. [Online]. Available: https://aclanthology.org/2025.emnlp-main.313.pdf

[11] D. Patterson *et al.*, "Carbon emissions and large neural network training," *arXiv preprint arXiv:2104.10350*, 2021. [Online]. Available: https://arxiv.org/abs/2104.10350

[12] A. S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of AI deployment?" in *Proc. ACM FAccT 2024*, pp. 85–99, 2024. [Online]. Available: https://dl.acm.org/doi/10.1145/3630106.3658542

[13] D. Pruthi, B. Dhingra, and Z. C. Lipton, "Combating adversarial misspellings with robust word recognition," in *Proc. ACL 2019*, pp. 5582–5591, 2019. [Online]. Available: https://aclanthology.org/P19-1561.pdf

[14] A. Q. Jiang *et al.*, "Mistral 7B," *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: https://arxiv.org/abs/2310.06825

[15] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Proc. NeurIPS 2022*, pp. 507–520, 2022. [Online]. Available: https://arxiv.org/abs/2207.08815

[16] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. NeurIPS 2022*, pp. 24824–24837, 2022. [Online]. Available: https://arxiv.org/abs/2201.11903

[17] Y. Zhu *et al.*, "PromptBench: Towards evaluating the robustness of large language models on adversarial prompts," *arXiv preprint arXiv:2306.04528*, 2024. [Online]. Available: https://arxiv.org/abs/2306.04528

[18] A. Aliakbarzadeh *et al.*, "Exploring robustness of multilingual LLMs on real-world noisy data," *arXiv preprint arXiv:2501.08322*, 2025. [Online]. Available: https://arxiv.org/abs/2501.08322

[19] M. Moradi and M. Samwald, "Evaluating the robustness of neural language models to input perturbations," in *Proc. EMNLP 2021*, pp. 1558–1570, 2021. [Online]. Available: https://aclanthology.org/2021.emnlp-main.117.pdf

[20] D. Romero-Alvarado, J. Hernandez-Orallo, and F. Martinez-Plumed, "How resilient are language models to text perturbations?" in *IDEAL 2024, LNCS*, vol. 15346, Springer, 2025. [Online]. Available: https://dl.acm.org/doi/10.1007/978-3-031-77731-8_8